React 1/2

With Vite and TypeScript

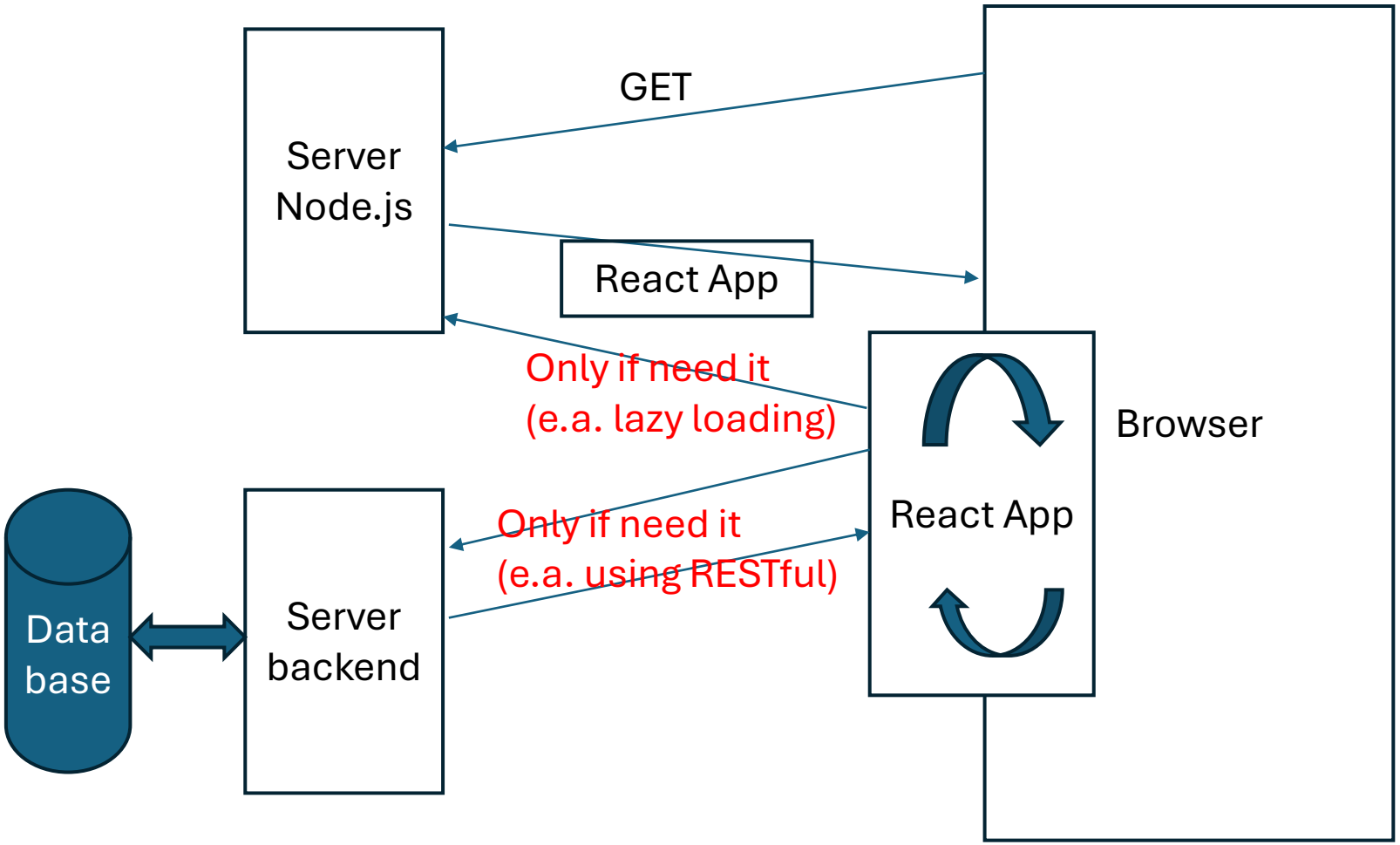
Syllabus

- SPA and MPA application
- Vite+React
  - Installation
  - Running
  - Operation
  - Extensions in VS Code
- The first React+Vite project
  - Project Structure
  - Getting Started
  - The meeting point of HTML and React
  - JSX
  - React's Virtual DOM Tree
- Demonstration project
  - Project Folder Structure
  - Layout
- TypeScript
  - Types in Notation
  - Custom Types
- Communication between components, changes in components
  - Props
  - States
  - Virtual DOM
  - Data Service

SPA and MPA application

How the SPA application works

- The operation of a SPA application (e.g., in React) is as follows.
  - All **subpages** are **within** the SPA application, **communicating via actions within** the browser.
  - Often, the addresses of these two servers differ only in the port number.



The idea of the SPA application

- 1) The application user enters a URL into the browser.
- 2) The web application server **sends the entire** application as JavaScript code (plus any other static files).
- 3) The browser-side code executes and creates the correct web page.
- 4) The user can click a link to another page **in the** application.
- 5) Then, instead of connecting to the server, the application executes the JavaScript code locally, creating the requested web page and optionally changing the URL in the browser's address field.
- 6) If creating a page requires dynamic data, it connects to another server (e.g., with a different port number) and retrieves only the data (e.g., in JSON format), which it inserts into the appropriate locations within the page being created.
- 7) This allows for faster response to page changes because the page is already in the browser from the start; the only additional time is the exchange of simple data files to populate the page.

Practical additions 1/2

- In **MPA** (**Multi-Page Application**) applications, each page is created from scratch, which takes time.
- The **SPA** (**Single Page Application**) approach theoretically requires all pages to be created in advance:
  - often saved in a special way, depending on the framework, e.g., .tsx files for React.
  - Creating valid HTML from these files is called **Client-Side Rendering** (**CSR**), as opposed to **Server-Side Rendering** (**SSR**),
    - such as Java Spring MVC + Thymeleaf.
- These pages can **all** be sent to the browser **at once**. However, for large applications, this would result in **long wait times** for the application to load. Hence, **Code Splitting** is used, where fragments of JS code are only loaded when the user enters a specific tab.

Practical additions 2/2

- The local routing mechanism described in point 5) is called **Client-Side Routing** (**SCR**) and is not added by default to React projects. However, any larger application will require subpages, so you should add such a library.
  - there are several of them; React Router will be selected.
- Point 6) represents the *separation of layers approach* (**Decoupling**), as the server providing logical/business data doesn't have to be the same as the server serving the data presentation layer (i.e., the React application). And it shouldn't be. This allows us to swap it out at any time, even using a different technology. This means the presentation layer will be in React, and the business layer:
  - in Java Spring,
  - in Node.js/Express.js,
  - in Python/Django,
  - in C#/ASP
  - etc.
- Layer separation also allows for the use of a different data server in the development or test/demo version.

Vite

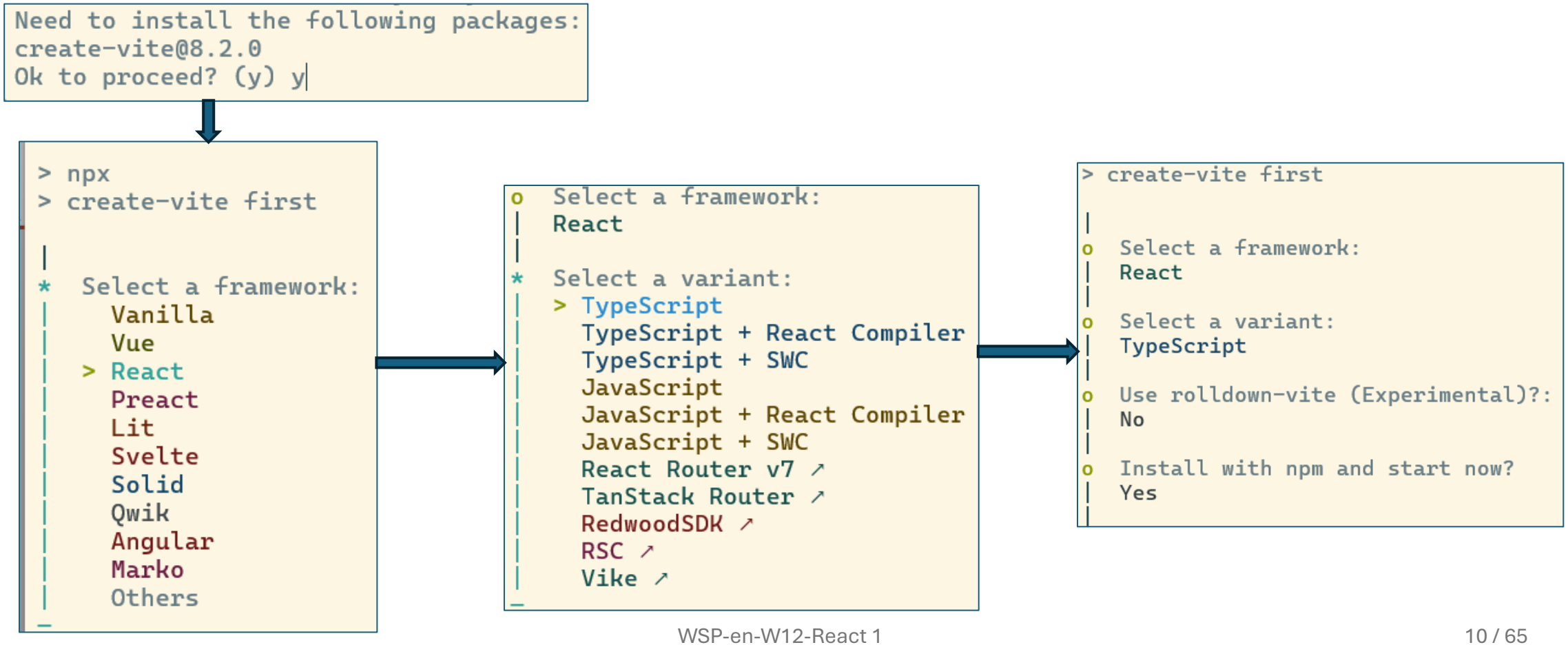
Installation
Running
Operation
Extensions in VS Code

Vite

- There's a tool for building various SPA (and other) projects using JavaScript:
  - React
  - Angular
  - Vue
  - Svelte
  - etc.
- The unmentioned project types are very good for the future, but to understand how React works and its logic, we won't use them.
- Projects can be written in either pure (Vanilla) JavaScript or the TypeScript superlanguage, which is ultimately compiled to JavaScript (because only such code works in browsers).
  - TypeScript allows for type safety at the compilation stage, and the prepared development environment then recognizes many programmer errors that are not treated as errors in Vanilla JavaScript.
  - The programming environment can suggest field/parameter names when it knows what type a given variable/parameter/function is.
  - For this reason, TypeScript will be used during the lecture.

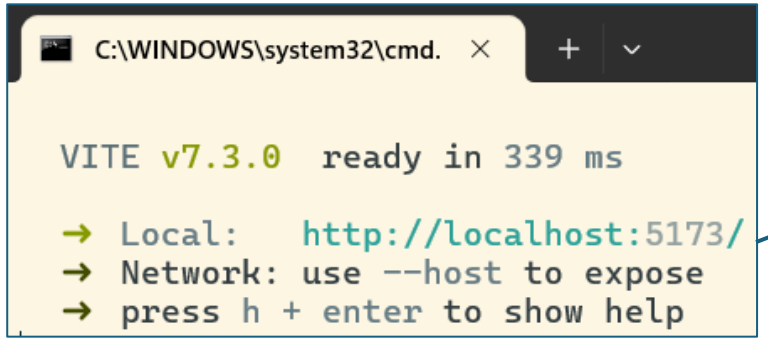
Vite - Installation

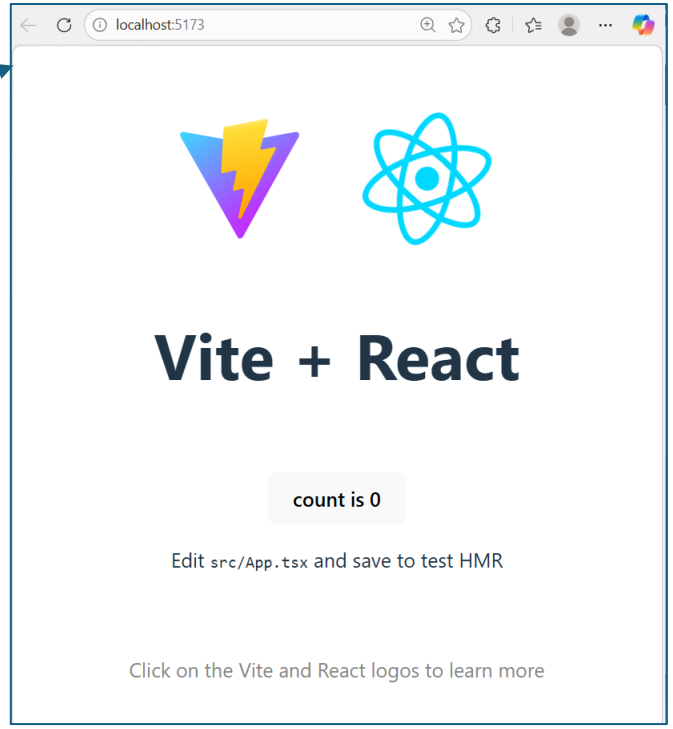
- Installation in the projects folder (this command will create subfolder "`first`"):
  - `npm create vite@latest first`
  - Select the React and TypeScript (superlanguage for JavaScript) options, without any experimental features.



End of installation - launching application

- Finally, Ctrl+left-click on the URL.
  - `q` - close the application.



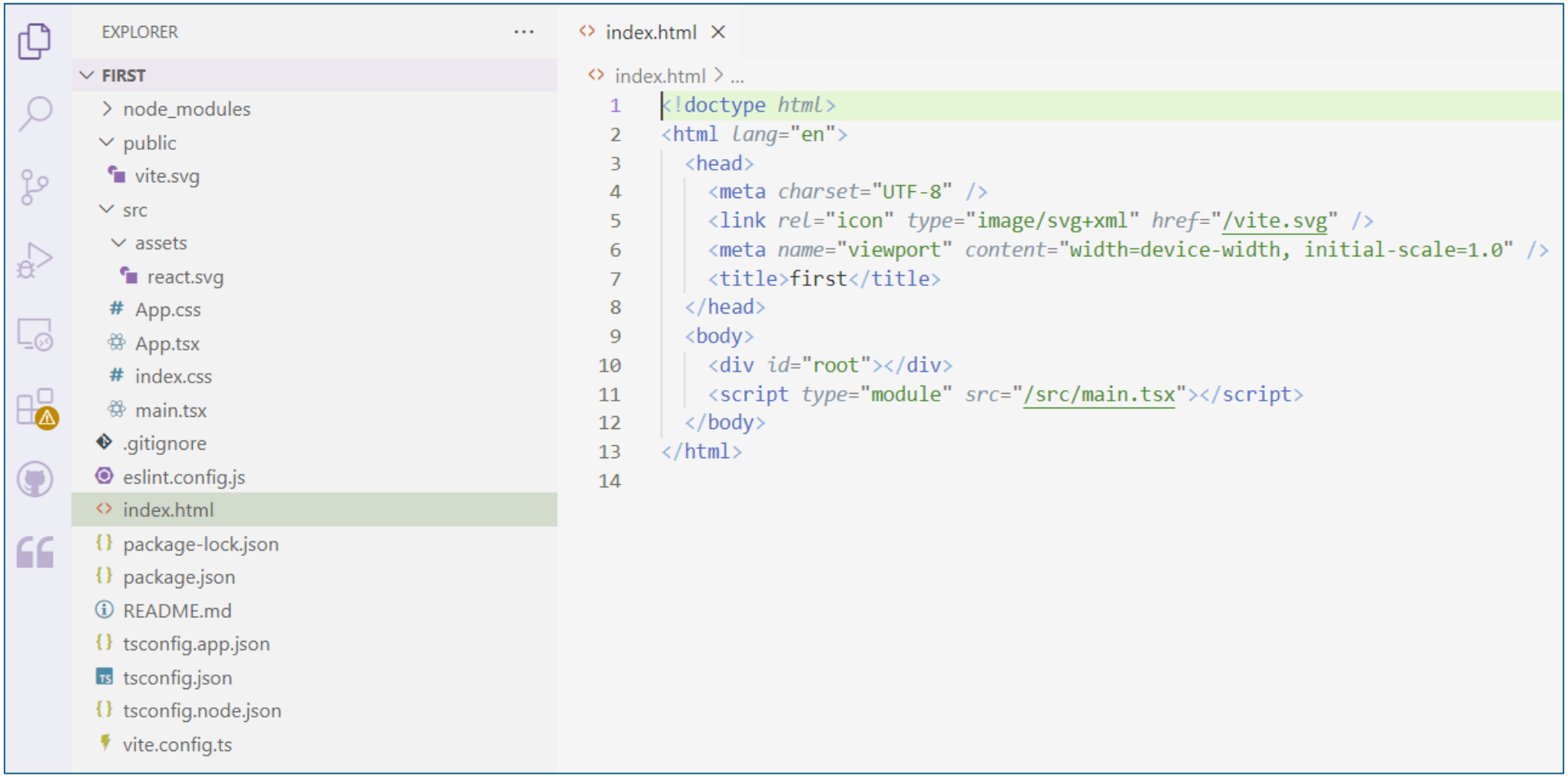



```
  VITE v7.3.0  ready in 289 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
h

  Shortcuts
  press r + enter to restart the server
  press u + enter to show server url
  press o + enter to open in browser
  press c + enter to clear console
  press q + enter to quit
q
```

```
\Wykład\W12>cd first

\Wykład\W12\first>code .
```

Project `first` in Visual Studio Code

- The starting point of the application is the `index.html` file



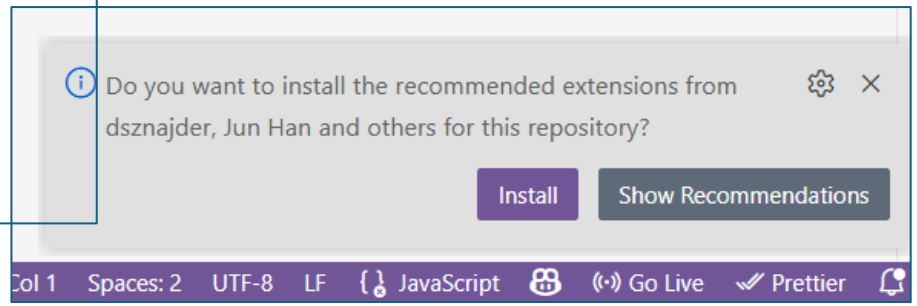
Extensions in VS Code

- For better, faster work in the Visual Studio Code environment, I recommend installing the following plugins:
  - **ESLint** – A basic tool for static code analysis.
    - Catches logical and syntactic errors (e.g., unused variables or errors in hooks) before the application runs.
  - **Prettier** – Code formatter – Automatically ensures consistent code formatting (indentation, semicolons, quotes).
    - Crucial for team projects, ensuring that each participant's code looks identical in the Git repository.
  - **ES7+ React/Redux/React-Native snippets** – A powerful database of shortcuts.
    - For example, typing 'rafce' and pressing Tab instantly generates a complete functional component skeleton with exports.
- It's also worth installing:
  - **Error Lens** – Makes TypeScript and ESLint errors "loud."
    - Displays error text directly in the code line, allowing to quickly respond to typos.
  - **Auto Rename Tag** – Automatically renames the closing tag when you edit the opening tag.
    - Extremely useful for refactoring large JSX blocks.
  - **Total TypeScript** – Acts as a "translator" for TypeScript errors.
    - Explains complex typing error messages in a way that's understandable for beginners (and others).
  - **Thunder Client** or **REST Client** – Lightweight alternatives to Postman built into VS Code.
    - Let you test your Spring Boot endpoints without leaving the editor.

Recommendation file for VS Code

- The plugins mentioned above can be created automatically. Simply navigate to (or create) the `.vscode` subfolder in your project folder and copy the `extensions.json` file below.
- Then reopen VS Code for that project.

```
{
  "recommendations": [
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "dsznajder.es7-react-js-snippets",
    "formulahendry.auto-rename-tag",
    "usernamehw.errorlens"
  ]
}
```



The first React+Vite project

Project Structure

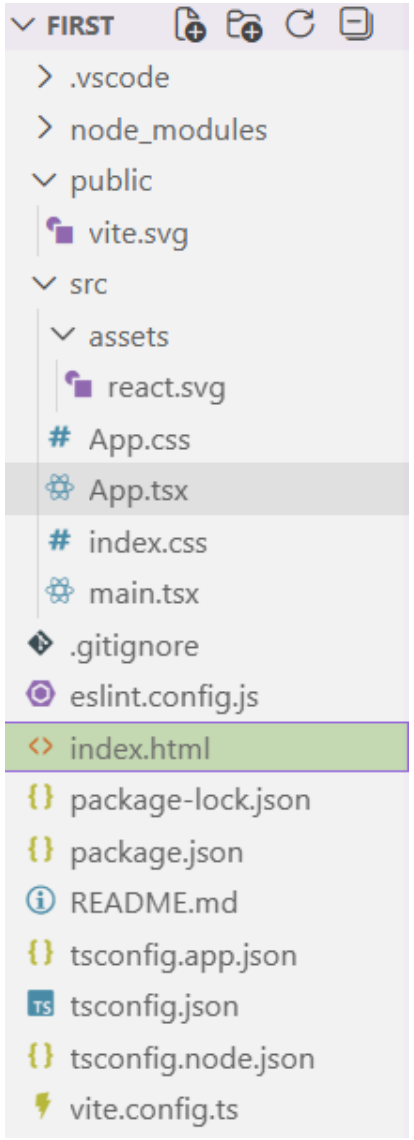
Getting Started

The meeting point of HTML and React

JSX

React's Virtual DOM Tree

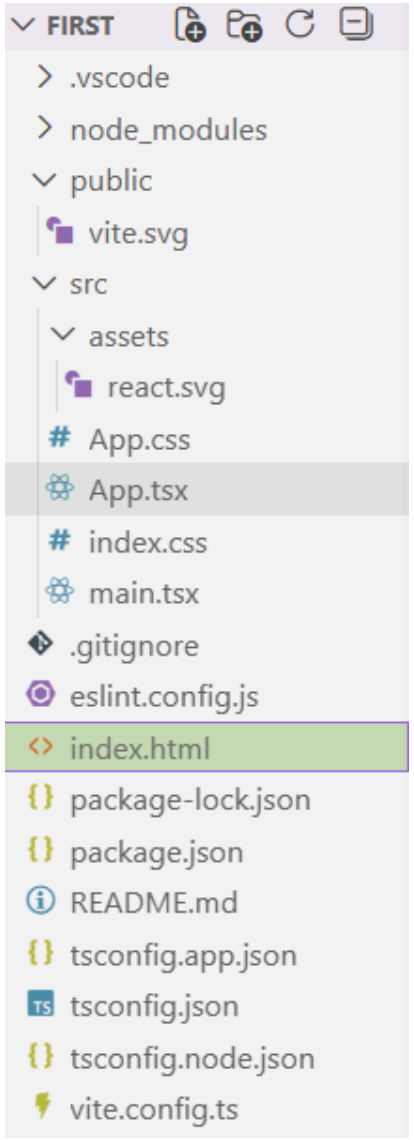
Project files 1/4



- "Administrative" files:
  - `package.json` – The most important configuration file (equivalent to `pom.xml`, `gradle.build`).
    - Contains a list of libraries, project versions, and scripts (e.g., dev, build).
  - `package-lock.json` – An automatically generated file that locks the exact versions of all dependencies.
    - Ensures that the project will run consistently on every computer.
  - `vite.config.ts` – Configuration file for Vite itself.
    - This is where we configure plugins (e.g., for React), server ports, and Spring proxies.
  - `.gitignore` – List of files ignored by Git (by default, it includes `node_modules` and builds).

- These files will **not be changed** for now.

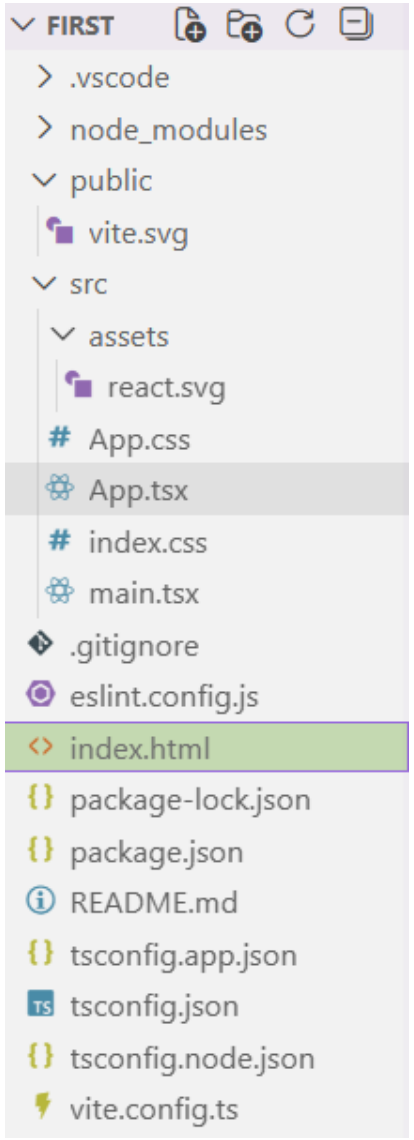
Project files 2/4



- Entry Point (Public & Root)
  - `index.html` – In Vite, this file is located in the **root** folder (not in `public`).
    - This is the "skeleton" into which React "injects" the entire application. It contains only one important element: `<div id="root"></div>`.
  - `public/` – Folder for static assets that don't change during compilation (e.g., `favicon.ico`).
    - Files here are accessible directly at `/filename`.

- These elements **will be changed or added** to as **needed**.

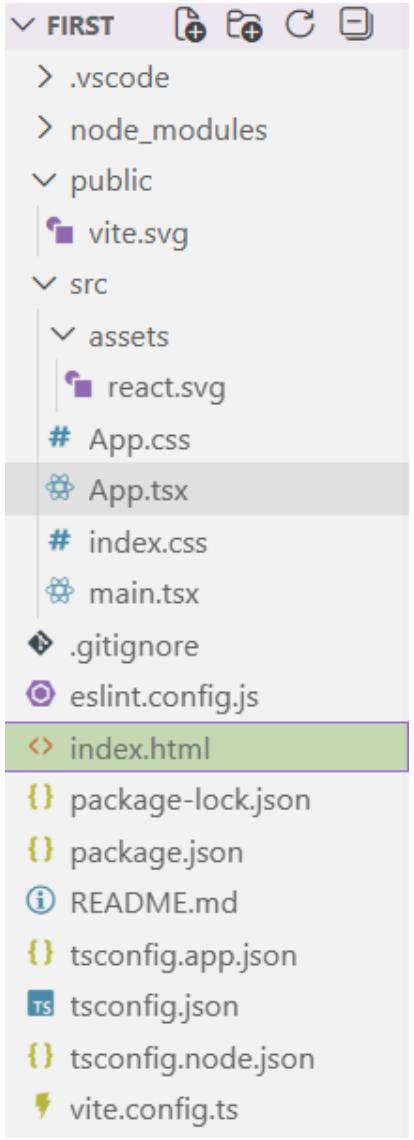
Project files 3/4



- Source Code (`src/` folder)
  - `main.tsx` – Entry point to the JS code.
    - This is where React looks for the `#root` element in HTML and assembles the application within it. This is the equivalent of the `public static void main()` method in Java.
  - `App.tsx` – The main component of your application.
    - In the React hierarchy, this is the "parent" of all other components that will be created later.
  - `App.css` and `index.css` – CSS styles.
    - Typically, `index.css` contains global styles (margin reset, fonts), while `App.css` contains styles specific to the main component.
  - `assets/` – Folder for graphics or icons that are imported directly into `.tsx` files (Vite optimizes them during build).

- This is the location that will be **primarily modified**.

Project files 4/4



- TypeScript Configuration
  - `tsconfig.json` and `tsconfig.app.json` – TypeScript compiler configuration.
    - Defines type checking rules (e.g., how strict the compiler should be).
  - `vite-env.d.ts` – File with Vite-specific type definitions
    - e.g., so that TS understands that we can import `.svg` files as modules.

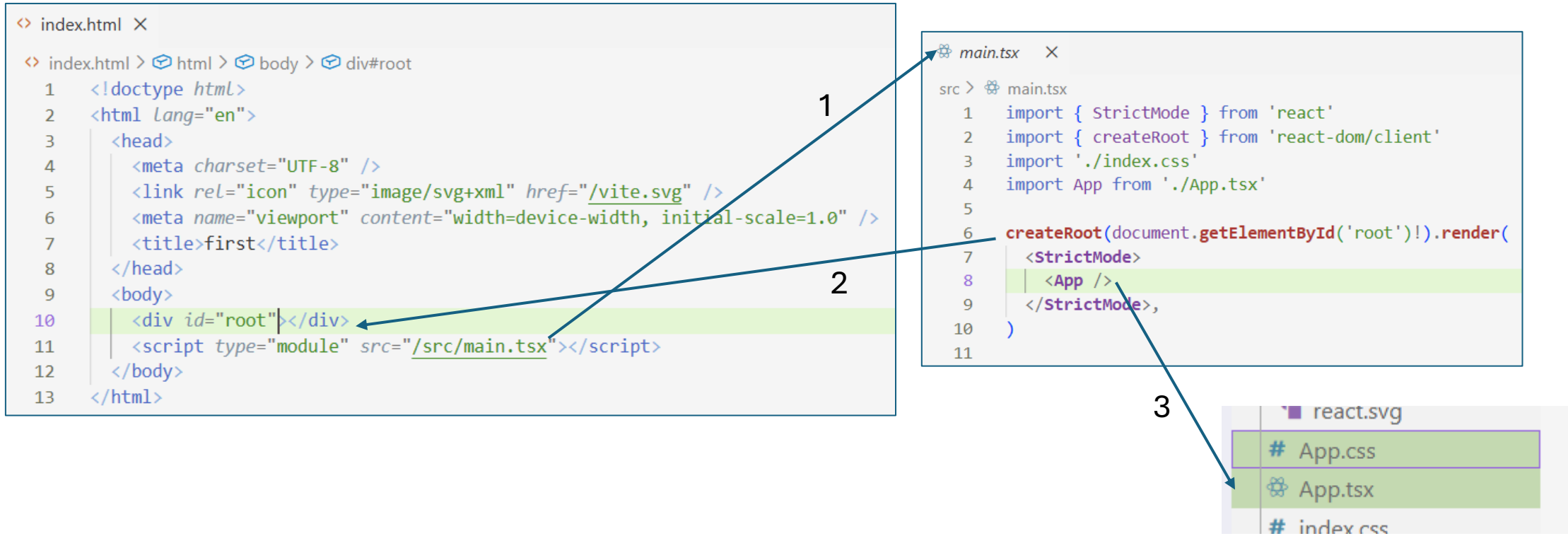
- These files will **not be modified**.

Launching the application

- The Vite system is a way to manage the **build**, **compilation**, and **execution** of a web application. Therefore, applications will most often be launched via NPM:
  - `npm run dev`
- In the development environment (`dev`), the **application** includes JS **code** that downloads small files (similar to project files) and **server-side code** that detects any file changes and sends them to the browser. This allows you to **change** the appearance and behavior of program components **on the fly**!
- To create the final production version, use the command:
  - `npm run build`
- which creates a distribution version in the `/dist` folder. NOTE: on a different port (e.g., 4173), to run it locally, you can run:
  - `npm run preview`
- This version **no longer has additional code** that replaces files on the fly, and the files are combined into **larger packages** that are sent to the browser once, whenever a package element is needed for current operation. Running `preview` is a fairly simple web server; it's better to run it through NPX (Node Package Execute) using:
  - `npx serve -s dist`
- The lecture will use `npm run dev`.

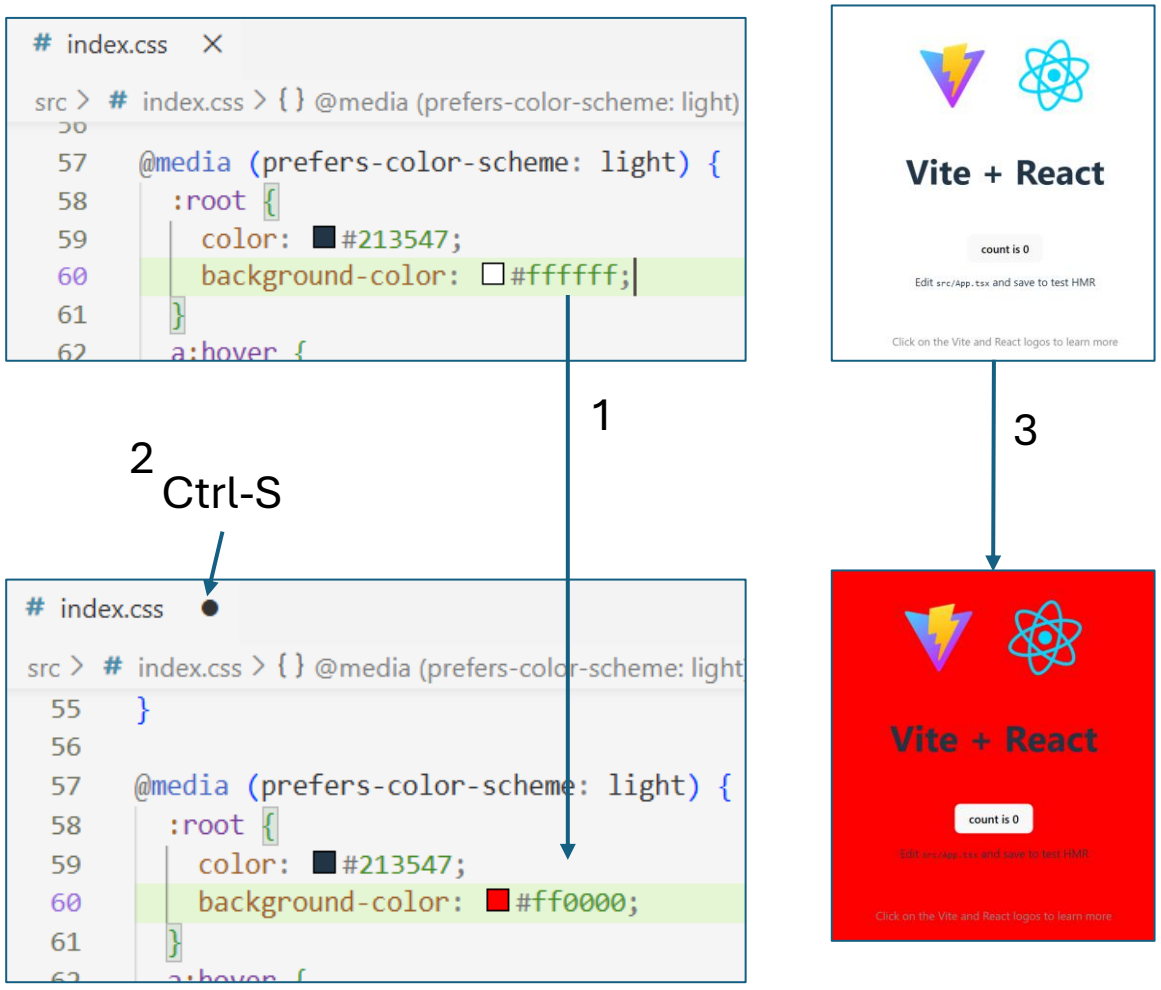
The meeting point of HTML and React

- 1) The application begins with the index.html file, which contains the `/src/main/tsx` script.
- 2) The meeting point of HTML and React is the element with the ID "`root`".
  - This could theoretically be a different element, but we won't change that.
- 3) The `App` element and any other elements created by the developer begin execution.
  - This means that control passes to the TypeScript code in the `App.tsx` file.



Where is CSS?

- Style files are treated specially. They are not entered directly into the `<head>` element of the `index.html` file.
- They are inserted differently throughout the application depending on the launch method (`dev` or `build`) – as described on the previous pages.
- They are inserted into the application/component via import, e.g.:
  - `import './index.css'`
  - `import './App.css'`
- It's worth testing how the `dev` version works by changing the page background color in `index.css` and saving the changes to disk.



Component and JSX

- The files below are after simplifying the application.
- Each component's function returns JSX (JavaScript XML), which is a kind of JavaScript+XML+HTML format (without the quotes/apostrophes!)
- Becuase `class` is a keyword for JS/TS, we use attribute `className` instead.

main.tsx

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

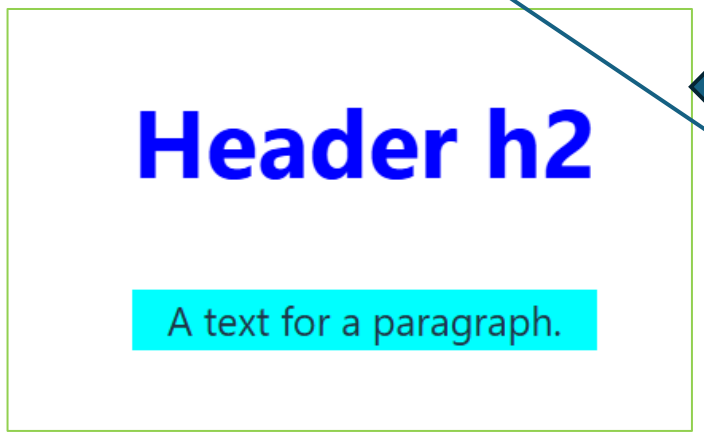
App.tsx

```tsx
import './App.css'

function App() {

  return (
    <div className="container" id="main-wrapper">
      <h2 style={{ color: 'blue' }}>Header h2</h2>
      <p>A text for a paragraph.</p>
    </div>
  );
}

export default App
```

JSX

App.css

```css
#root {
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}
h2{
  font-size: 2.4em;
}
p{
  background-color: aqua;
}
```



JSX

- If we want to insert JS/TS code in JSX, we enclose it in braces: `{code}`. The next brace is simply a JS/TS object: `{{objectData}}`.
- The JSX result is converted into a virtual DOM element within React. The compiler calls the creation of this DOM element for the developer.
  - using `React.createElement(type, props, ...children)`
- Here's how it's translated during React/Vite compilation:

App.tsx

src > App.tsx > ...

```tsx
import './App.css'

function App() {

  return (
    <div className="container" id="main-wrapper">
      <h2 style={{ color: 'blue' }}>Header h2</h2>
      <p>A text for a paragraph.</p>
    </div>
  );
}

export default App
```

```tsx
return React.createElement('div', { className: 'container', id: 'main-wrapper' },
  React.createElement('h2', { style: { color: 'blue' } },'Header h2'),
  React.createElement('p', null, 'A text for a paragraph.')
);
```

From JSX to HTML

- JSX is essentially **syntactic sugar**.
  - It's currently **standard** JavaScript.
- The returned result must be a **single element**,
  - so if we have multiple components, we must insert them, for example, in a `<div>...</div>` or even in a `<>...</>`.
- The `<>...</>` element cannot have any attributes.
  - This is shortcut for the `<React.Fragment>...</React.Fragment>` element, which in its full form can have a `key` attribute.
  - When converting these components to valid HTML, wrapper HTML elements will not be created.

```
// Error! This won't compile.
return (
  <h1>Tytuł</h1>
  <p>Treść</p>
);
```

```
return (
<div>
  <h1>Tytuł</h1>
  <p>Treść</p>
</div>
);
```

final HTML

```
<div>
  <h1>Tytuł</h1>
  <p>Treść</p>
</div>
```

```
return (
<>
  <h1>Tytuł</h1>
  <p>Treść</p>
</>
);
```

final HTML

```
  <h1>Tytuł</h1>
  <p>Treść</p>
```

React's virtual DOM

- React's virtual DOM is similar to a DOM tree, but it can contain elements that are React components created by the developer. Therefore, it contains **two types** of elements:
  - DOM elements (**lowercase**): e.g., `<div>`, `<h1>`, `<button>`. React knows to simply draw them in the browser.
  - Components (**uppercase**): e.g., `<StudentList />`, `<Navbar />`. These are your own, defined functions that also return other elements "underneath."
- This allows for a better understanding of the structure of our page (React code on the left) than the pure HTML (on the right):

```
<Header />
<Sidebar />
<StudentTable />
```

```
<div class="header">...</div>
<div class="sidebar">...</div>
<div class="content-table">...</div>
```

Demonstration project

Project Folder Structure

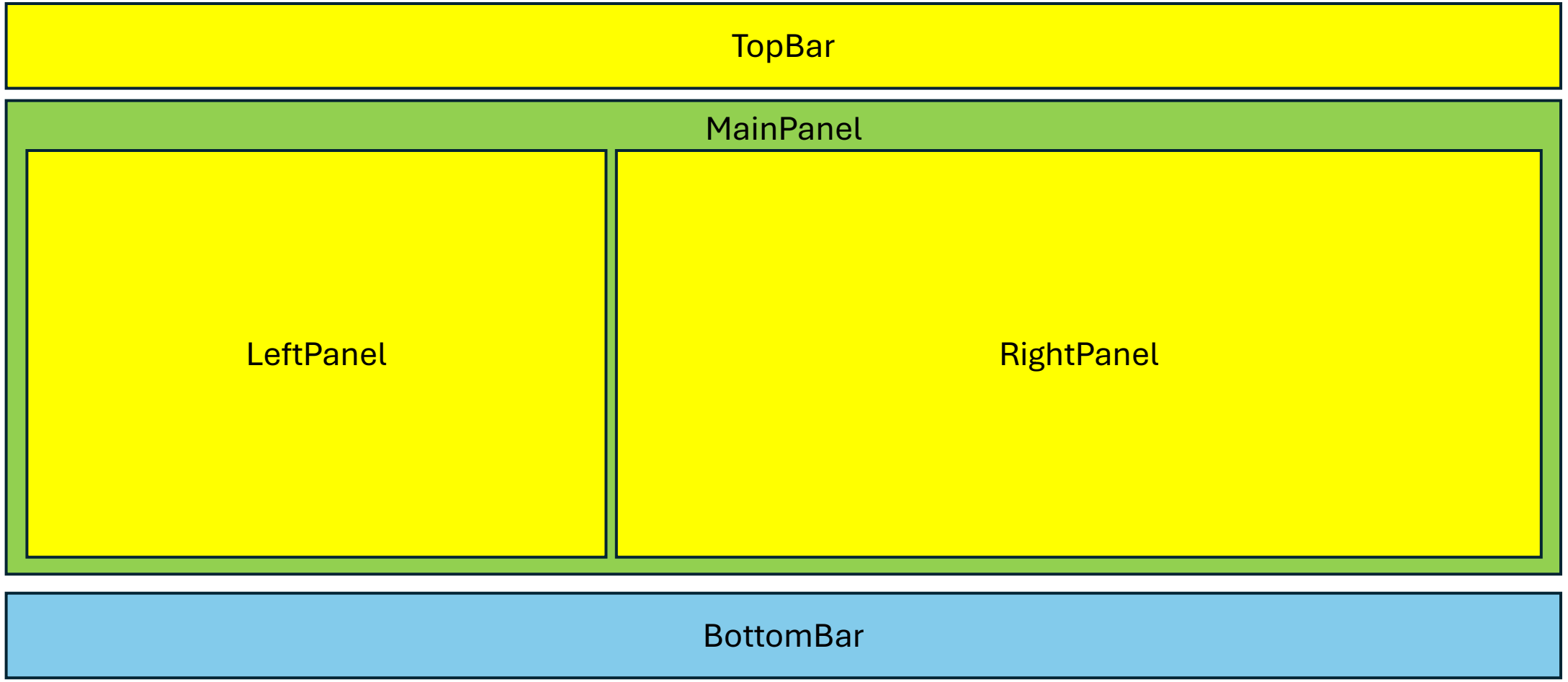
Layout

Project Folder Structure

- React **doesn't** enforce a folder structure, but it's worth separating elements with **the same functionality**: components, services, and type definitions. This allows for easier navigation and subsequent changes. For example:
  - `src/components/`
    - views or components (e.g., `UserTable.tsx`, `UserRow.tsx`).
  - `src/services/`
    - e.g., `userService.ts` (first with mocks, then with fetch).
  - `src/models/`
    - business type definitions (`Student`, `Course`).
  - `src/types/`
    - definitions of additional TypeScript types (e.g., `interface User { id: number... }`).

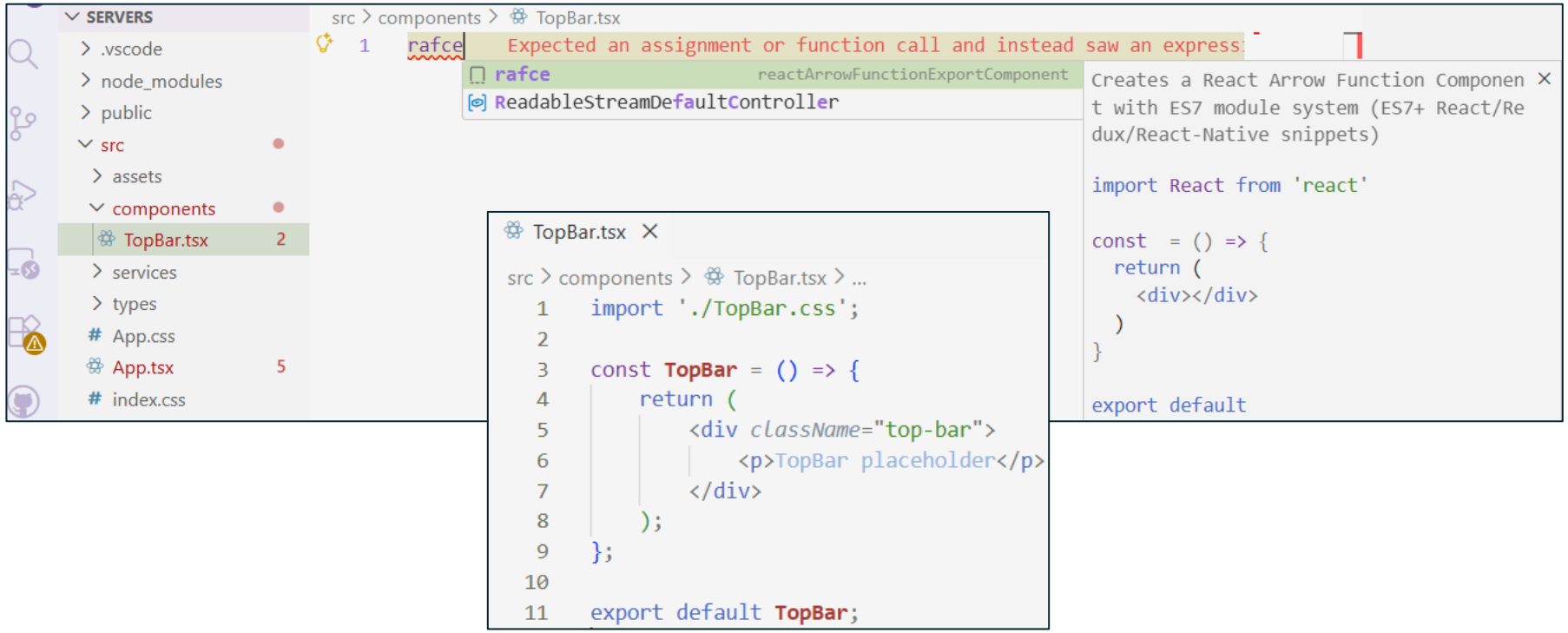
Project - Layout

- Using components, you can design your own view layout for your application.
- For the following proposal, you'll need to create 5 components.



Automatic code generation

- If you have the **ES7+ React/Redux/React-Native snippets** extension installed, simply create a `TopBar.tsx` file, enter "`rafce`" in the file, and press <*Enter*>.
- For now, you can delete the first line of code and add the CSS import instead.
- We also add an empty `TopBar.css` (it's even better to do this first).
- Similarly, create the BottomBar, `LeftPanel`, and `RightPanel`.



Files for MainPanel and App

- Below are the functions for the `App` and `MainPanel` components, reflecting the relationships between components.
- Neither has a placeholder, as we assume they contain other components that will fill the entire surface.

src > App.tsx > ...

```tsx
import './App.css'
import BottomBar from './components/BottomBar'
import MainPanel from './components/MainPanel'
import TopBar from './components/TopBar'

function App() {
  return (
    <div className="app-container">
      <TopBar />
      <MainPanel />
      <BottomBar />
    </div>
  );
}

export default App
```

```tsx
import LeftPanel from "./LeftPanel"
import RightPanel from "./RightPanel"
import "./MainPanel.css"

const MainPanel = () => {
  return (
    <div  className="main-panel">
        <LeftPanel />
        <RightPanel />
    </div>
  )
}

export default MainPanel
```

How does React see our CSS styles?

- Import Mechanism
  - By writing `import './LeftPanel.css'`, we **don't isolate** styles within the component.
  - We only tell the build tool (Vite) to include this file in the final page stylesheet.
- "Common Scope" (Global Scope)
  - All imported CSS files are merged into **one large** file.
  - Problem: If you use the `.button` class in `TopBar.css` and `BottomBar.css`, both components will have the same appearance (**the last** imported file wins!).
- Golden Rule: Unique Selectors
  - To avoid conflicts, we prefix classes with the component name.
  - Instead of: `.container { ... }`
  - We use: `.left-panel-container { ... }`
  - Alternatively: `.left-panel { ... }`

Final layout

- After proper styling, we can achieve the following effect.

src > components > # LeftPanel.css > ...

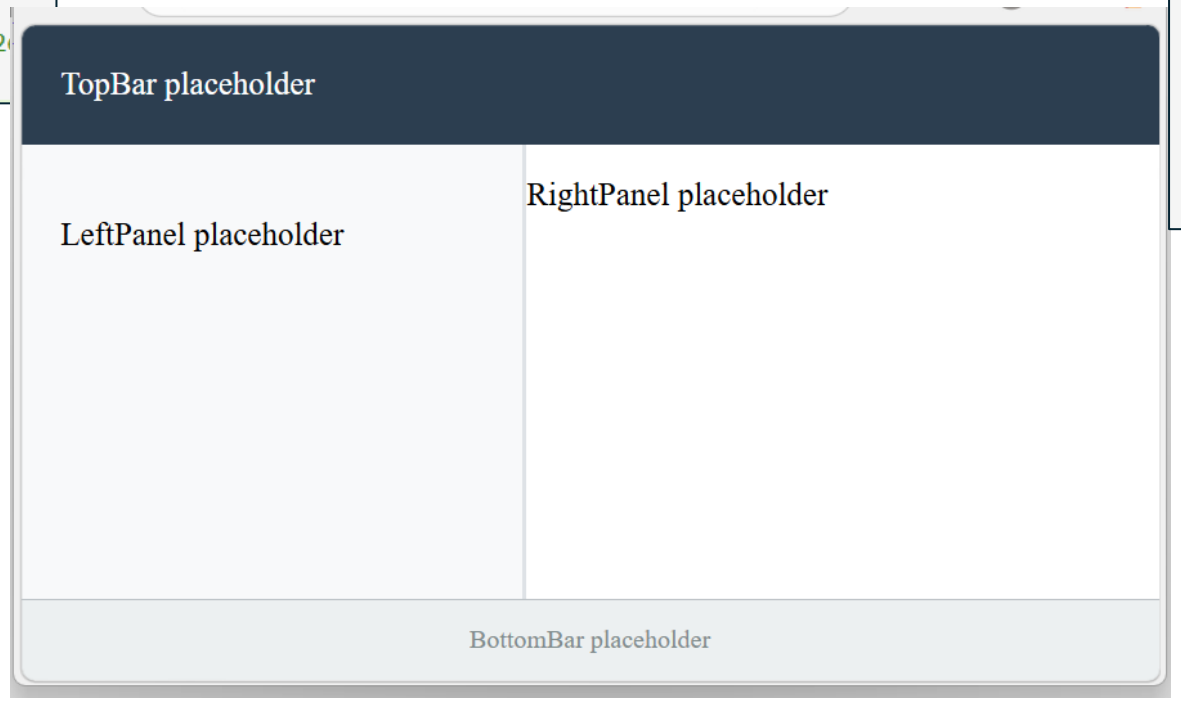
```css
.left-panel {
  width: 40%;
  background-color: #f8f9fa;
  padding: 20px;
  border-right: 2px solid #dee2
}
```

src > components > # MainPanel.css >

```css
.main-panel {
  display: flex;
  flex: 1; /* Wypełnij r
}
```

src > components > # BottomBar.css > ...

```css
.bottom-bar {
  height: 40px;           /* Nieco
  background-color: #ecf0f1; /*
  color: #7f8c8d;
  display: flex;
  align-items: center;
  justify-content: center; /* Wyśr
  font-size: 0.8rem;
  border-top: 1px solid #bdc3c7;
  flex-shrink: 0;         /* Zapobi
}
```



src > components > # TopBar.css > ...

```css
.top-bar {
  height: 60px;           /* Stała
  background-color: #2c3e50; /*
  color: white;
  display: flex;
  align-items: center;    /* Wyśro
  justify-content: space-between;
  padding: 0 20px;
  flex-shrink: 0;         /* Ważne
}
```

src > components > # RightPanel.css > ...

```css
.right-panel {
  width: 60%;
  background-color: #ffffff;
}
```

TypeScript

Types in Notation

Custom Types

Custom data types in TypeScript

- The main difference between Typescript and JavaScript is defining an fixed data type for variables, parameters, etc.
- In notation, this means placing **a colon after** the **identifier** and specifying the type.
- The type can be specified directly or via the type name.
- Directly: e.g.
  - `let result: undefined | "ok" | "error";`
- Create a type identifier and use it, e.g.
  - `type ResultType = undefined | "ok" | "error";`
  - `let result: ResultType.`
- In the above case, the `undefined` keyword means that the variable (etc.) can have an undefined value, either the string value `"ok"` or `"error"`. Any other value assigned to such a variable will cause a **compilation** error,
  - but not necessarily at runtime, because that's where JavaScript works.
- Since there may often be a need to use, for example, an initial undefined value, there is syntactic sugar for such cases. You use a question mark `'?'` **after the variable/field** name if it can also have an undefined value. So the above codes can be written differently (better):
  - `let result?: "ok" | "error";`
  - `type ResultType = "ok" | "error";`
  - `let result?: ResultType;`
  - `let knownResult: ResultType; // cannot be undefined`

Defining types - examples

- A type can be defined as:
  - an interface
  - a union of types
  - a class

```
interface User {
  id: number;
  name: string;
  email: string;
  isActive: boolean;
}
```

```
type Status = 'active' | 'inactive' | 'suspended';
type Ident = string | numer;
```

```
class Product {
  constructor(
    public id: number,
    public name: string,
    public price: number
  ) {}

  getFormattedPrice(): string {
    return `$${this.price.toFixed(2)}`;
  }
}
```

Defining types

- Definition of the `Student`, `Department`, and `Course` types.
- **Interfaces** are **preferred** over classes, but for example, the `Course` **class** will do.

src > models > TS Department.ts > ...

```ts
export type Department = "W1" | "W2" | "W3" | "W4N";
```

src > models > TS Student.ts > ...

```ts
import type { Department } from "./Department";

export interface Student {
    id: number;
    index: number;
    firstName: string;
    lastName: string;
    department:Department;
    icon:string;
}
```

src > models > TS Course.ts > ...

```ts
export class Course {
    id: number;
    name: string;

    constructor(id: number, name: string) {
        this.id = id;
        this.name = name;
    }
}
```

```ts
export interface Course {
    id: number;
    name: string;
}
```

Communication between components, changes in components

Props
States
Virtual DOM
Data Service

Communication between components

- The most common situation is the need for communication between components that have a **parent-child** relationship.
- First, we'll present the downward communication, i.e., from **parent to child**.
- Let the `App` component read the current time and pass this information to the `TopBar` component.
- Similarly, `App` should pass the name of the university as a parameter.
- For this purpose, so-called **props** (*properties*) are used.
- Data stored **as HTML attributes** is sent to the child element (`TopBar`).
  - `time={currentTime}`
  - `university={universityName}`
- In the child element itself, all props are received as **a single object** with **fields named like** the HTML **attributes**.
  - `const TopBar = ({ time, university }: TopBarProps)`
- Therefore, in TypeScript, it's worth creating an appropriate type for props in the child component's file.
  - `interface TopBarProps {`

App code modification

- Data stored **as HTML attributes** is sent to the child element (`TopBar`).

```
function App() {
  const universityName = "Wroclaw University of Science and Technology";

  const currentTime= new Date().toLocaleTimeString();

  return (
    <div className="app-container">
      <TopBar
        time={currentTime}
        university={universityName}
      />
      <MainPanel />
      <BottomBar />
    </div>
  );
}
```

Parameter as one object

- You can receive props as one object with a chosen name (e.g. `props`) and then use the **dot notation**:

src > components > TopBar.tsx > ...

```tsx
import './TopBar.css';

// We define a simple type/interface for clarity
interface TopBarProps {
  time: string;
  university: string;
}

const TopBar = (props: TopBarProps) => {
    return (
        <div className="top-bar">
            <div className="university-name">
            {props.university}
        </div>
        <div className="system-info">
          Logged at: {props.time}
        </div>
          </div>
    );
};

export default TopBar;
```

Parameter with destructuring

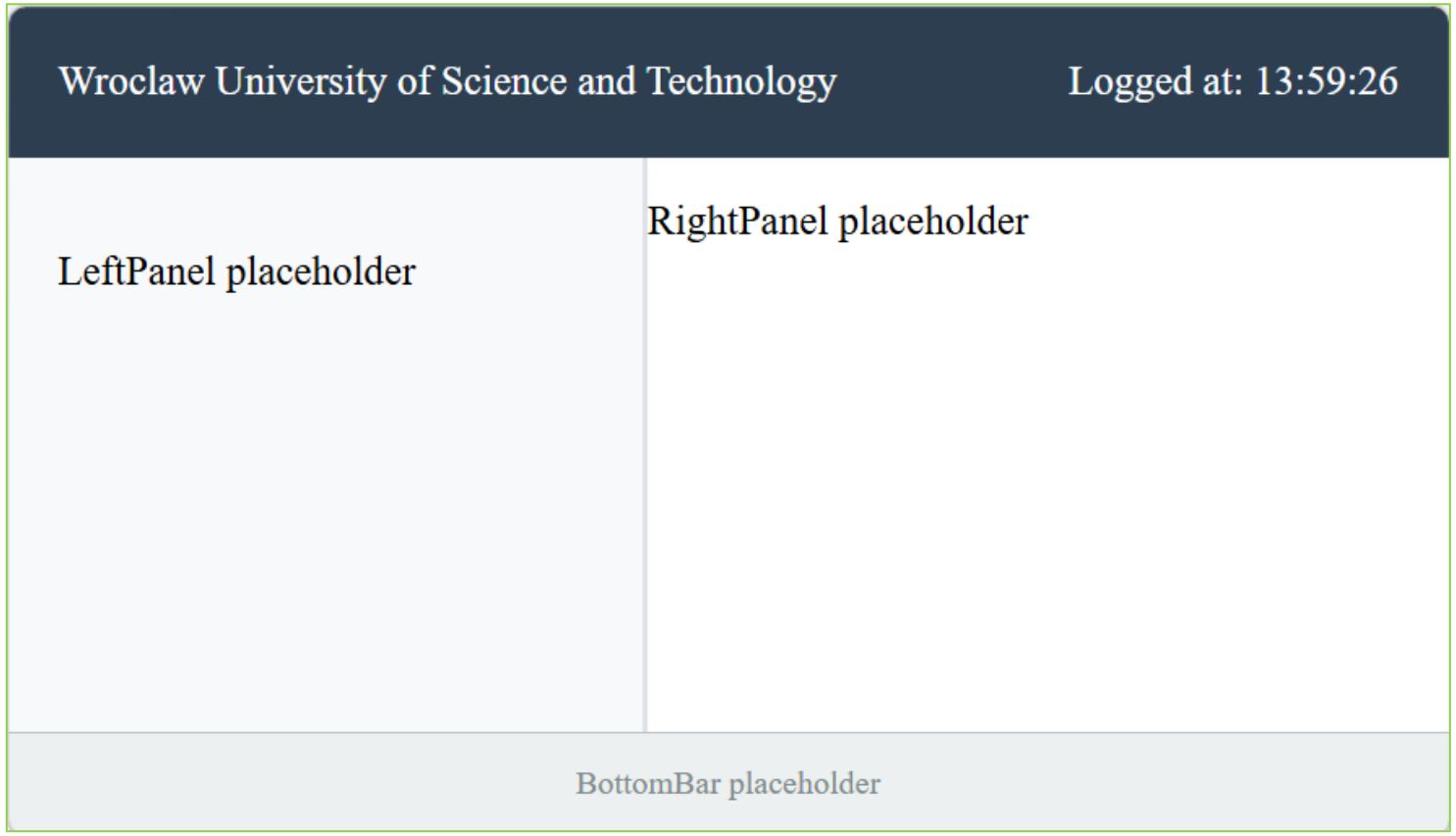
- It is worth using the option of "unpacking" (**Destructuring**) the object into fields (they **must** have **the same names**).

src > components > TopBar.tsx > ...

```tsx
import './TopBar.css';

// We define a simple type/interface for clarity
interface TopBarProps {
  time: string;
  university: string;
}

const TopBar = ({ time, university }: TopBarProps) => {
    return (
        <div className="top-bar">
            <div className="university-name">
        {university}
      </div>
      <div className="system-info">
        Logged at: {time}
      </div>
        </div>
    );
};

export default TopBar;
```

Effect

- Each screen refresh (reload `index.html`) updates the time.



Local time refresh

- Reloading means **launching** (or perhaps even reloading from the server) our **entire application** and rebuilding the entire website.
- We'd like to force a time update, but using a button in the TopBar component **without launching** the **entire** application.
- Props are read-only; they cannot be changed.
- The correct method is to **inform the parent** that an action has occurred in the child that requires a response, such as updating props.
- To make this possible, two mechanisms are implemented:
  - states
    - for storing data that may change in the component,
  - callbacks (callback functions)
    - for informing the parent what happened.

States

- States are a type of hook.
- States are an element of a component.
- It's essentially a **one-dimensional array** of hooks (states), but we won't be using them through indexes.
- When creating a new state in code, we usually destruct the array (Array Destructuring) on the left side of the '=' sign.
- In the state constructor itself, we insert an initial value.
- A state is actually a two-element array, where:
  - the **zeroth** element is the constant **stored value**.
  - the **first** element is the **function for changing** this value, but the change occurs **asynchronously**, so not immediately. This also **forces** the **regeneration of the component** in which the value changed!

State - notation, array destructuring

- States must be imported :
  - `import { useState } from 'react';`
- The top notation used is array destructuring.
  - preffered way
- The bottom one shows how it is decomposed into single assignments.

```
// 1. Define State: [value, function_to_change_value]
const [currentTime, setCurrentTime] = useState(new Date().toLocaleTimeString());
```

```
const state = useState(new Date().toLocaleTimeString());
const currentTime = state[0];
const  setCurrentTime = state[1];
```

Props extension for `TopBar`

- To receive feedback, you need to add a callback function declaration to the props (`TopBarProps`) interface, e. a.: `onRefresh()`
  - Add it to the props in the `TopBar` function definition
- And in the returned JSX, add a button and call this function when it's pressed: `onClick={onRefresh}`.

```tsx
// We define a simple type/interface for clarity
interface TopBarProps {
  time: string;
  university: string;
  onRefresh: () => void; // A function that takes no arguments and returns nothing
}
```

```tsx
const TopBar = ({ time, university, onRefresh }: TopBarProps) => {
    return (
        <div className="top-bar">
            <div className="university-name">
          {university}
        </div>
        <div className="system-info">
          <span>Time: {time}</span>
          <button onClick={onRefresh} style={{ marginLeft: '10px' }}>
            Refresh Time
          </button>
        </div>
          </div>
    );
};
```

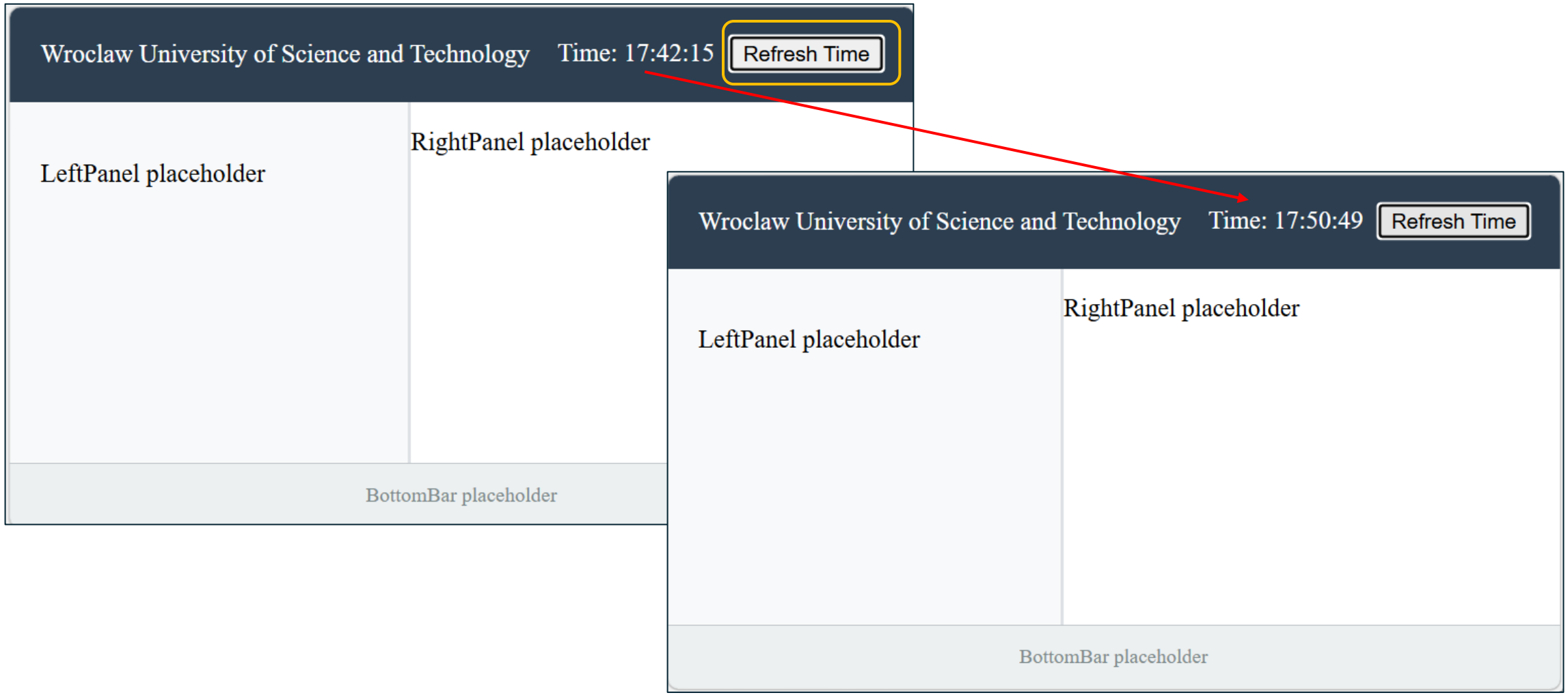
Adding state and features to the App

- The `handleRefreshTime` function updates the current time using the state setter (`setCurrentTime`).
- It must be added to the `TopBar` element's attributes.

```
function App() {
  const universityName = "Wroclaw University of Science and Technology";

// 1. Define State: [value, function_to_change_value]
  const [currentTime, setCurrentTime] = useState(new Date().toLocaleTimeString());

  // 2. Define the action to be performed
  const handleRefreshTime = () => {
    setCurrentTime(new Date().toLocaleTimeString());
  };

  return (
    <div className="app-container">
      <TopBar
        time={currentTime}
        university={universityName}
        onRefresh={handleRefreshTime}
      />
      <MainPanel />
      <BottomBar />
    </div>
  );
}
```

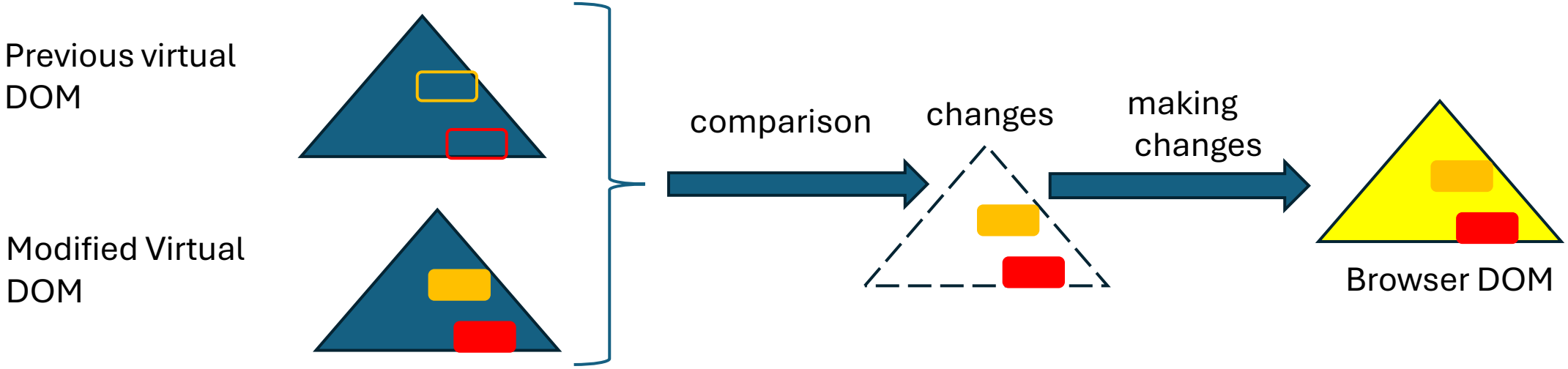
The effect

- When you click the button, the time refreshes **without reloading** the page.



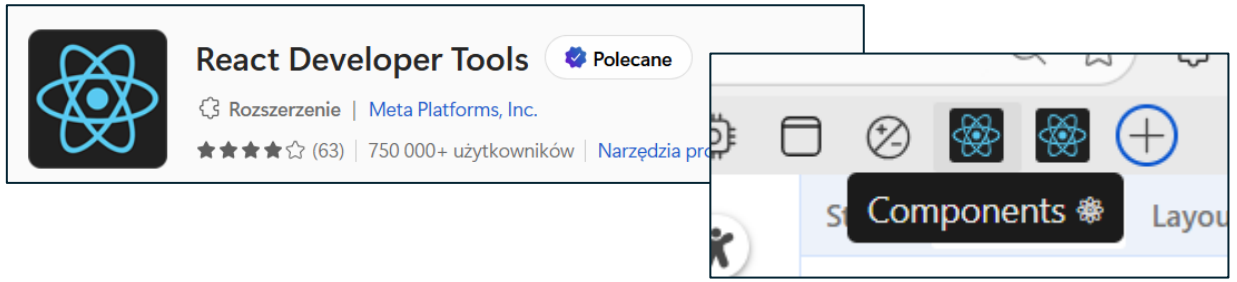
Virtual DOM

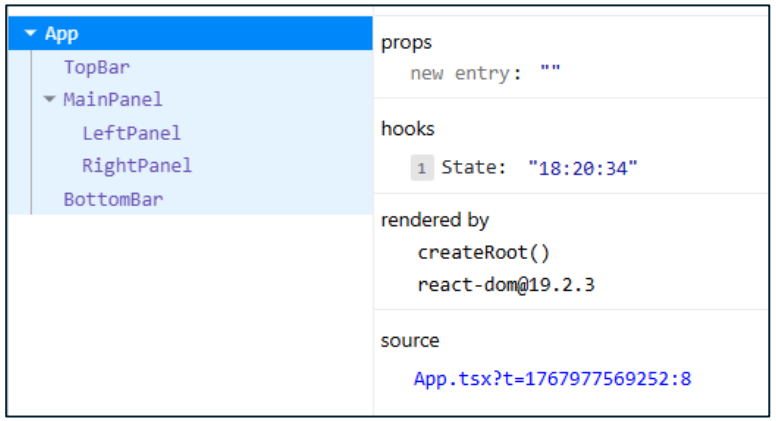
- By making a change to the time-related state, we force the App component and its child components to be recreated.
  - However, this **only** happens **once**, as **all states** in the application will **first change** and then trigger a reprocessing of the components, starting from the topmost component in the virtual DOM tree.
  - However, the browser DOM tree will only change minimally. React remembers the **previous** state of **its virtual** DOM tree **and** the **changed state** and compares what actually changed. It only makes the **necessary changes** to the browser DOM tree. In our case, the `<span>Time: {time}</span>` element (which is why it's worth separating it into a small `<span>` element) has a text (content) change, and **only this change** will be reflected in the browser DOM tree.



React development tool

- When working in React, it is worth installing **React Developer Tools in your browser** (https://react.dev/learn/react-developer-tools).
- Reset your browser. Open DevTools, the "Components" tab with the React icon.
- This will show how React sees the page:
  - components (tree),
  - props,
  - states (as hooks),
  - and other information.



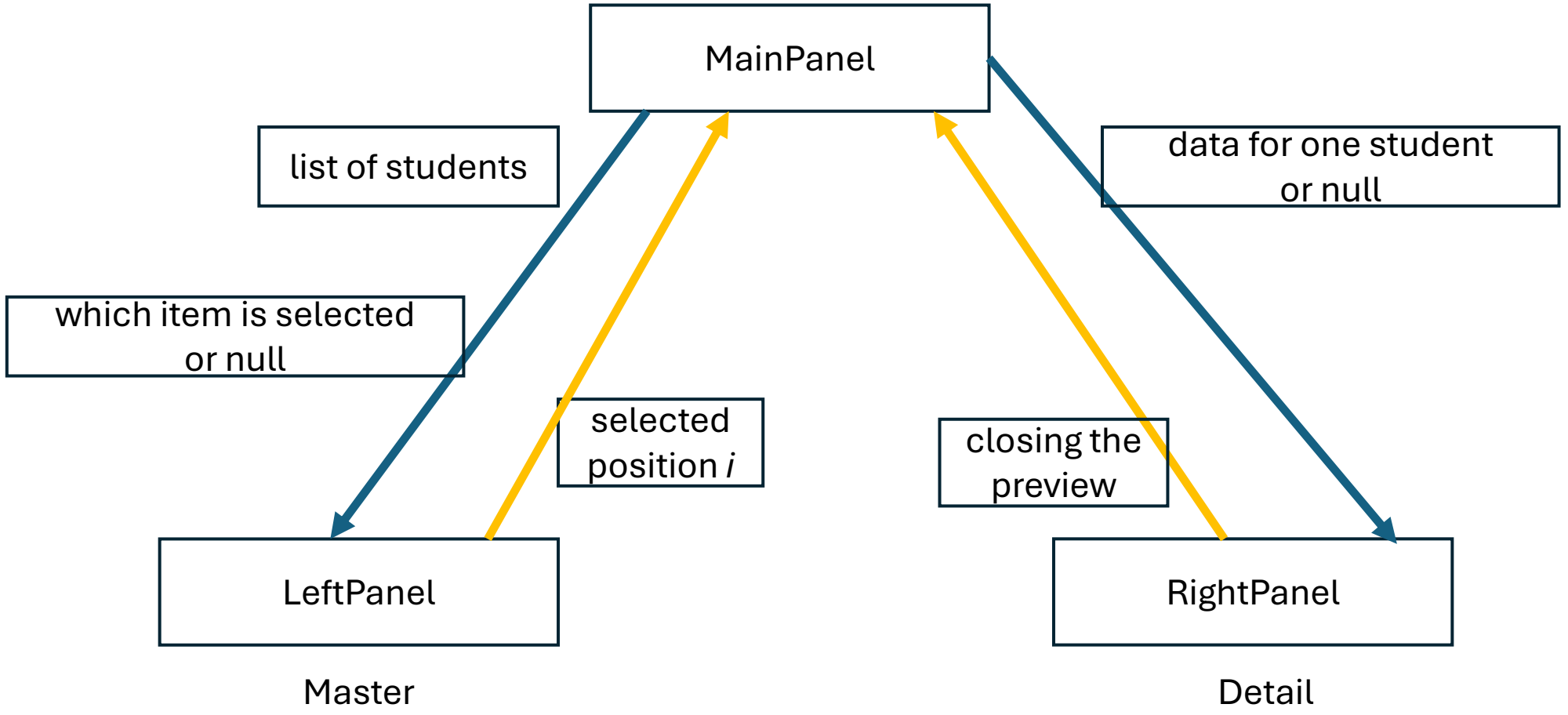


Presentation of the student list

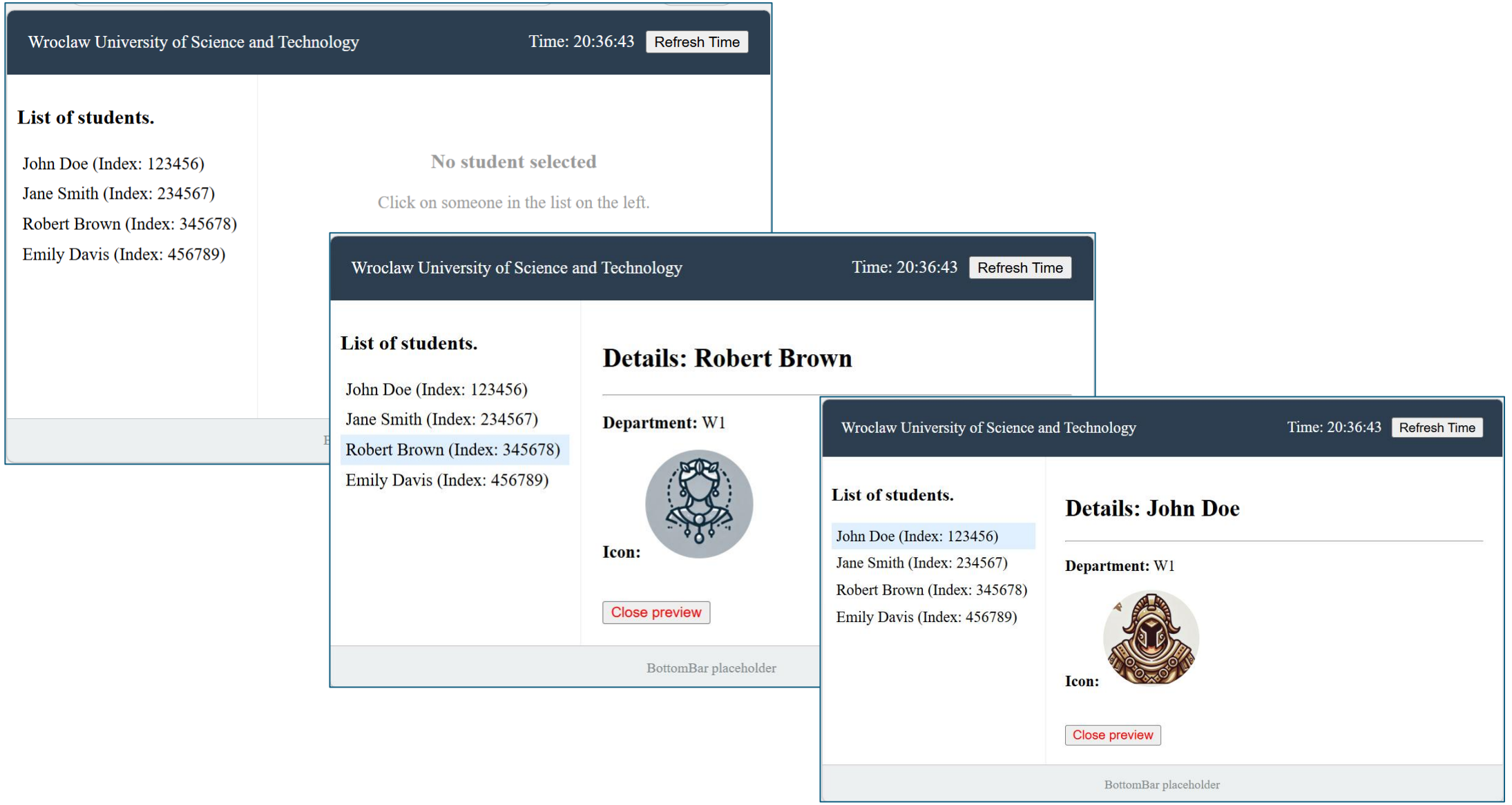
- Let's say the left panel displays a list of students (name and surname), and after clicking on a student, the right panel displays more student data.
- The selected item should be highlighted in the left panel.
- A button should then close the data preview at the bottom of the right panel. The highlighted item should then disappear. The right panel should also indicate that no data has been selected.
- You can also select a different student if one was previously selected.
- Access to the data should be managed by the `MainPanel`.

Data and Actions

- An example of the Master-Detail Pattern from the User Interface (UI) design patterns group.



Expected effect



Data service

- To make it easier to download data from any source in the future, let's add a layer of services responsible for it :
  - A class `UniversityServis` with a method `getStudents()`.

rvices > TS UniversityService.ts > ...

```ts
import type { Student } from "../models/Student";

const MOCK_STUDENTS: Student[] = [
    { id: 1, index: 123456, firstName: "John", lastName: "Doe", department: "W1", icon: "/icons/icon1.png" },
    { id: 2, index: 234567, firstName: "Jane", lastName: "Smith", department: "W2", icon: "/icons/icon2.png" },
    { id: 3, index: 345678, firstName: "Robert", lastName: "Brown", department: "W1", icon: "/icons/icon3.png" },
    { id: 4, index: 456789, firstName: "Emily", lastName: "Davis", department: "W4N", icon: "/icons/icon4.png" },
];

export const UniversityService = {
    getStudents: (): Student[] => {
        return [...MOCK_STUDENTS];
    }
};
```

Changes to `MainPanel` - code

- Now the `MainPanel` needs to retrieve student data, determine which one is selected (or none), and inform the left and right panels about it.

```
4   import { UniversityService } from "../services/UniversityService"
5   import { useState } from "react"
6
7   const MainPanel = () => {
8     const [students] = useState(UniversityService.getStudents()); // Data from the "service"
9     const [selectedId, setSelectedId] = useState<number | null>(null);
10
11    // We calculate a specific student object based on its ID
12    // This is better than maintaining two states because we have a "single source of truth"
13    const selectedStudent = students.find(s => s.id === selectedId) || null;
14
15    const handleSelect = (id: number) => setSelectedId(id);
16    const handleDeselect = () => setSelectedId(null);
17
18    return (
19      <div  className="main-panel">
20        <LeftPanel
21          students={students}
22          selectedId={selectedId}
23          onSelect={handleSelect}
24        />
25        <RightPanel
26          student={selectedStudent}
27          onClear={handleDeselect}
28        />
29      </div>
30    )
31  }
```

MainPanel Changes - Explanation

- Since we won't be changing the student list within the MainPanel at this time, we only need a getter from the state we'll remember, so we only need to "extract" the zeroth element of the array:
  - `const [students] = useState(UniversityService.getStudents());`
  - The left panel will need this data.
- The next state will remember and modify which student was selected by their ID (or none, which will be null).
  - `const [selectedId, setSelectedId] = useState<number | null>(null);`
  - This information will be needed by both the left and right panels.
- The next information needed for the right panel is the selected student's data (the entire Student object) or null. This isn't a new state, just a simple variable, as it depends on the previous state.
  - `const selectedStudent = students.find(s => s.id === selectedId) || null;`
- The `handleSelect` and `handleDeselect` functions are callback functions for the left and right panels, respectively, informing which student has been selected or that the data preview window has been closed and a student needs to be deselected. Finally, three of the above data are passed to the left panel and two to the right.

How the `useState()` method works

- Why is it better to use a state in this case than a regular variable for an immutable collection of students?
  - That is: `const [students] = useState(UniversityService.getStudents());`
  - Not: `const students = UniversityService.getStudents();`
- The idea is to execute the component (here: `MainPanel`) multiple times and maintain the `[value] = useState(initValue)` method.
  - Before each call to the `MainPanel()` method, the **component's state/hook counter** is set to 0.
  - When `MainPanel()` is first executed, upon encountering the `useState()` method, it checks to see if a state exists for this component at the counter position (here: 0). There isn't one. Therefore, a new state is created and initialized with the `initValue` value. Then, the state counter is incremented by one. When the next `useState()` occurs in the code, it will create a state at position 1, etc.
  - The next time `MainPanel()` is executed, when the `useState()` method is encountered, it again checks whether there is a state at the counter position for this component (in this case: 0). This time, there is! So it **just returns it**, increments the state counter and exits!
- This means that reading students from the service will ONLY occur the **first time** the `MainPanel()` method is used.
  - In the case of a regular variable, reading from the service would occur **each time** the `MainPanel()` method is executed. This approach can be useful if this is exactly what we want.
- The same applies to the `[selectedId, setSelectedId]` state. It will be at position 1 of the state array and will only be set to null the first time it's executed.

Left panel - code

```tsx
import type { Student } from '../models/Student'

interface LeftPanelProps {
  students: Student[];
  selectedId: number | null;
  onSelect: (id: number) => void;
}

export function LeftPanel({ students, selectedId, onSelect }: LeftPanelProps) {
  return (
    <div style={{ width: '30%', padding: '10px', borderRight: '1px solid #eee' }}>
      <h3>List of students.</h3>
      <ul style={{ listStyle: 'none', padding: 0 }}>
        {students.map(s => (
          <li
            key={s.id}
            onClick={() => onSelect(s.id)}
            style={{
              padding: '5px',
              cursor: 'pointer',
              backgroundColor: s.id === selectedId ? '#e0f0ff' : 'transparent'
            }}
          >
            {s.firstName} {s.lastName} (Index: {s.index})
          </li>
        ))}
      </ul>
    </div>
  );
}
```

- The left panel receives a list of students, the selected student's ID (or `null`), and a method to call when a student is selected (this information is a function parameter).
- In this implementation, students are presented as an unordered list `<ul>`, where selection is performed by clicking the mouse (`onClick`).
- Creating multiple `<li>` fragments is accomplished using mapping and lambda expressions.

Left panel - explanation of event handling

- Two lines with attributes for elements created in the loop require additional explanation.
- The line `onClick={() => onSelect(s.id)}` - the `onClick` attribute always requires **a method** with **no arguments**. Therefore, we insert a definition for a method with no arguments that calls the single-argument `onSelect()` method.
  - More precisely: the attribute associated with the event requires a **reference** to a zero-argument method. Previously, for `TopBar`, we had a method with no arguments, `onRefresh()`, so the code only contained the name (reference) to this method: `onClick={onRefresh}`.
    - Theoretically, you could write `onClick={() => onRefresh()}`.
  - **Calling** these methods is **an error**, i.e., writing:
    - `onClick={onSelect(s.id)}`
    - `onClick={onRefresh()}`
    - These codes will return the result of the method (i.e. nothing, `void`) where a function reference is expected.

Left panel - explanation of the key attribute

- The `key={s.id}` line.
- The `key` attribute will appear primarily in lists, etc., especially if the elements can change (in this case: whether they are selected or not), their order, the number of elements, etc.
- Why is `key` necessary?
- React logic: React is incredibly fast because it **doesn't refresh the entire list** when a single element changes. To do this, it **needs to know which** specific element on the screen corresponds to which object in the `students` array.
- Imagine we have a list of 3 students and suddenly a new one is added to the top.
  - Without `key`: React sees: "Oh, the first element has changed, the second has changed, the third has changed, and there's a fourth." It **reworks everything**.
  - With `key`: React sees: "key=1, key=2, and key=3 are unchanged; they've **just moved down**. I **just** need to **draw a new element** with key=4 at the beginning."

Notes on key selection

- The Iron Rules of Key Selection
  - It must be **unique within the list**: There can't be two elements with the same key.
  - It must be **stable**: It must not change with each render (which is why `key={Math.random()}` is **the worst idea** in the world – components will "flick" and lose focus).
  - It should be derived from the data: It's best to use an ID from the database or a PESEL number/index.
- The most common mistake: Using the array index
  - `students.map((s, index) => <li key={index}>...)`
  - **only for static lists that never change**, don't sort, and don't filter.

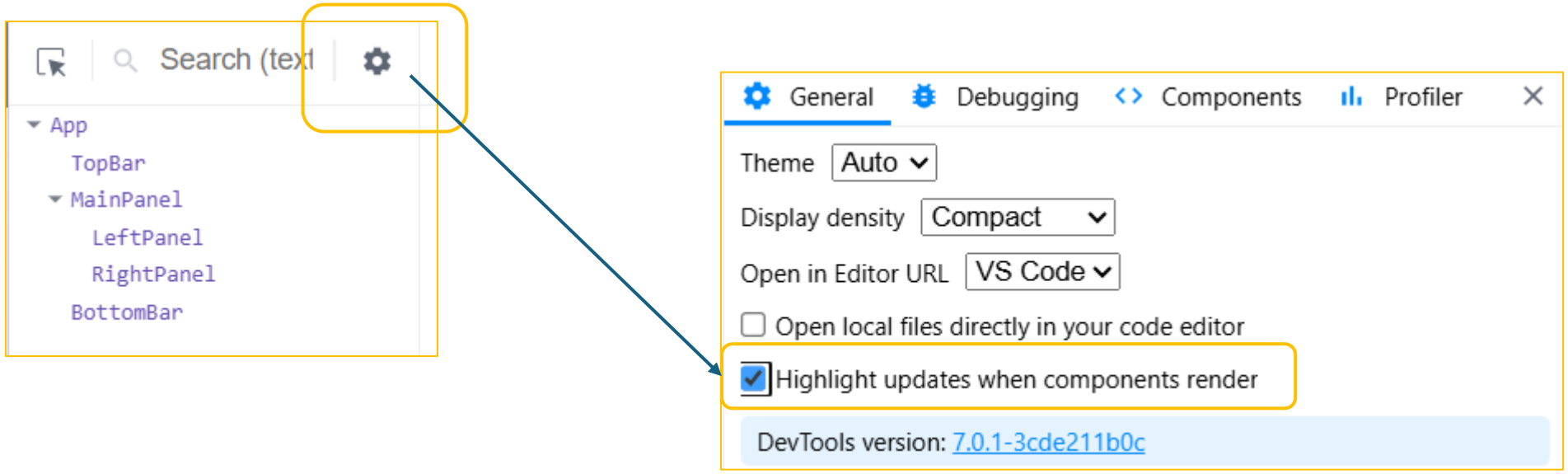
Right Panel - Code

```tsx
import type { Student } from '../models/Student'

interface RightPanelProps {
  student: Student | null;
  onClear: () => void;
}

export function RightPanel({ student, onClear }: RightPanelProps) {
  return (
    <div style={{ flex: 1, padding: '20px' }}>
      {!student ? (
        <div style={{ color: '#999', textAlign: 'center', marginTop: '50px' }}>
          <h3>No student selected</h3>
          <p>Click on someone in the list on the left.</p>
        </div>
      ) : (
        <div>
          <h2>Details: {student.firstName} {student.lastName}</h2>
          <hr />
          <p><strong>Department:</strong> {student.department}</p>
          <p><strong>Icon:</strong> <img
            src={student.icon}
            alt={`Ikona użytkownika ${student.firstName} ${student.lastName}`}
            style={{ width: '100px', height: '100px', borderRadius: '50%' }}
            />
          </p>
          <button onClick={onClear} style={{ marginTop: '20px', color: 'red' }}>
            Close preview
          </button>
        </div>
      )}
    </div>
  );
}
```

- In the right panel, we check whether we have `null` (in which case we return a placeholder – the code in the orange box) or whether we have student data. Then we create HTML with details about the student and a button to close the preview.
- This time, for the `onClick` event, we immediately provide a reference to a no-argument method (callback).

Addition 1

- Since in the current version the program changes several components with mouse clicks, it is worth checking the option below to make it easier to see which components have changed.



Addidtion 2

- If you have a project **without** `node_modules` folder (e.a. loaded from Git or from zip file), you have to run in the console (in the folder with the project):
  - `npm install`
- and needed modules will be loaded from internet.